

AXI-Lite FIR DSP Peripheral

Modular VHDL IP core implementing a register-controlled
FIR DSP interface with scalable architecture.

Verification results

Author: Gabriela Cabrera
Github: @MGabrielaCabrera
Date: March 2026
Version: 1.0

1.Verification description

This document presents the results of the initial verification phase for the VHDL modules developed in this project. The primary objective of this phase is to perform a first functional validation of each module individually through simulation.

The verification has been carried out using VHDL testbenches that rely on assertion-based checks to validate the expected behavior of the design. These assertions allow the simulation to automatically detect functional mismatches and ensure that the implemented logic behaves according to the intended specification.

For this first verification stage, each module has been tested independently. The document includes simulation waveform captures and relevant observations for each test case in order to illustrate the correct operation of the modules under different conditions.

It should be noted that this verification stage focuses on basic functional validation. In a later phase of the project, the verification environment will be extended using more advanced methodologies, including SystemVerilog-based testbenches, functional coverage, and a more structured verification framework. These improvements will enable a more comprehensive validation of the design and increase confidence in the correctness and robustness of the implementation.

2.AXI-Lite Slave Interface Verification

Module Description

The AXI-Lite Slave Interface module implements a memory-mapped interface compliant with the AXI4-Lite protocol. Its main function is to translate AXI-Lite bus transactions into a simplified internal register interface used by the rest of the design.

The module handles both read and write transactions through the standard AXI-Lite channels: write address, write data, write response, read address, and read data. Internally, the interface generates control signals such as register write enable, register read enable, register addresses, and write data for the connected register file.

Additionally, the module performs basic protocol checks, including address alignment verification and address range validation. When an invalid or misaligned address is detected, the module returns an AXI-Lite slave error response (SLVERR).

Testbench Architecture

The verification of the AXI-Lite Slave Interface module is performed using a dedicated VHDL testbench. The testbench instantiates the design under test (DUT) and emulates the behavior of an AXI-Lite master.

A clock generator provides the system clock with a period of 10 ns, while a reset sequence initializes the DUT before the start of the tests. The testbench includes a simple register file model composed of six 32-bit registers representing the memory-mapped registers accessible through the AXI-Lite interface.

The register file reacts to the internal control signals generated by the DUT:

- `reg_write_en` triggers register updates.
- `reg_read_en` enables register read operations.

Two procedures are implemented to simplify the stimulus generation:

- `axi_write`: performs a complete AXI-Lite write transaction including address, data, and response handshake.
- `axi_read`: performs a complete AXI-Lite read transaction including address and read data handshake.

These procedures allow the testbench to generate multiple test scenarios while respecting the AXI-Lite handshake protocol.

Test Scenarios

A set of directed tests has been implemented using assertion-based verification. Each test stimulates a specific aspect of the AXI-Lite protocol behavior and checks the correctness of the module using VHDL assertions.

The following test scenarios are executed:

1. **Basic Write Transaction:** A write operation is performed to register 0. The test verifies that the write response is OKAY and that the register file is updated with the expected value.
2. **Write to Another Valid Register:** A write operation is issued to register 1 to confirm correct address decoding and register update behavior.
3. **Read Transaction Verification:** The value previously written to register 0 is read back and compared with the expected data.
4. **Read from Another Register:** A read operation is performed on register 1 to verify correct readback functionality.
5. **Partial Write Using Byte Strokes:** A write transaction using partial byte enables (``WSTRB``) verifies that only selected bytes of the register are updated.
6. **Misaligned Write Address:** A write transaction is attempted using a misaligned address. The expected response is an AXI-Lite slave error (``SLVERR``).
7. **Out-of-Range Write Address:** A write is issued to an address outside the valid register range. The module should return a ``SLVERR`` response.

8. **Misaligned Read Address:** A read transaction using a misaligned address verifies that the module detects alignment errors.
9. **Out-of-Range Read Address:** A read operation targeting an invalid address verifies correct error handling.
10. **Back-to-Back Write Transactions:** Multiple consecutive writes are issued to different registers to verify correct operation under continuous traffic.
11. **Write Followed by Immediate Read:** A write operation is immediately followed by a read to confirm that the updated data can be retrieved correctly.

Simulation Results

The simulation results confirm the correct functional behavior of the AXI-Lite Slave Interface module for all tested scenarios.

All transactions follow the expected AXI-Lite handshake protocol. Valid write transactions correctly update the internal register file, and subsequent read operations return the expected data values. The byte strobe functionality correctly enables partial register updates according to the specified write mask.

Error handling mechanisms were also verified. Misaligned and out-of-range addresses correctly trigger AXI-Lite slave error responses ('SLVERR'), demonstrating that the implemented address validation logic functions as intended.

The accompanying waveform captures illustrate representative transactions, including valid read and write operations, byte strobe behavior, and error responses for invalid accesses.

```
Analyzing design files
Elaborating axi_lite_slave_if testbench
Running axi_lite_slave_if_tb testbench. Waveforms saved to waves/waves_axi_lite_slave_if_tb.fst
axi_lite_slave_if_tb.vhd:231:9:@50ns:(report note): Test 1: Write to register 0 and verify response
axi_lite_slave_if_tb.vhd:241:9:@90ns:(report note): Test 2: Write to register 1 and verify response
axi_lite_slave_if_tb.vhd:251:9:@140ns:(report note): Test 3: Read back register 0 and verify data
axi_lite_slave_if_tb.vhd:262:9:@220ns:(report note): Test 4: Read back register 1 and verify data
axi_lite_slave_if_tb.vhd:275:9:@300ns:(report note): Test 5: Write with partial byte strobes (upper two bytes only)
axi_lite_slave_if_tb.vhd:285:9:@400ns:(report note): Test 6: Write to misaligned address -> expect SLVERR
axi_lite_slave_if_tb.vhd:293:9:@450ns:(report note): Test 7: Write to out-of-range address -> expect SLVERR
axi_lite_slave_if_tb.vhd:301:9:@500ns:(report note): Test 8: Read from misaligned address -> expect SLVERR
axi_lite_slave_if_tb.vhd:309:9:@580ns:(report note): Test 9: Read from out-of-range address -> expect SLVERR
axi_lite_slave_if_tb.vhd:317:9:@660ns:(report note): Test 10: Back-to-back writes to consecutive registers
axi_lite_slave_if_tb.vhd:331:9:@810ns:(report note): Test 11: Write then immediate read (reg 3)
axi_lite_slave_if_tb.vhd:342:9:@990ns:(assertion note): End of simulation - all tests passed
Opening waveform viewer (only if the fst file exists)
Test complete!
```

Figure 1 – Test results

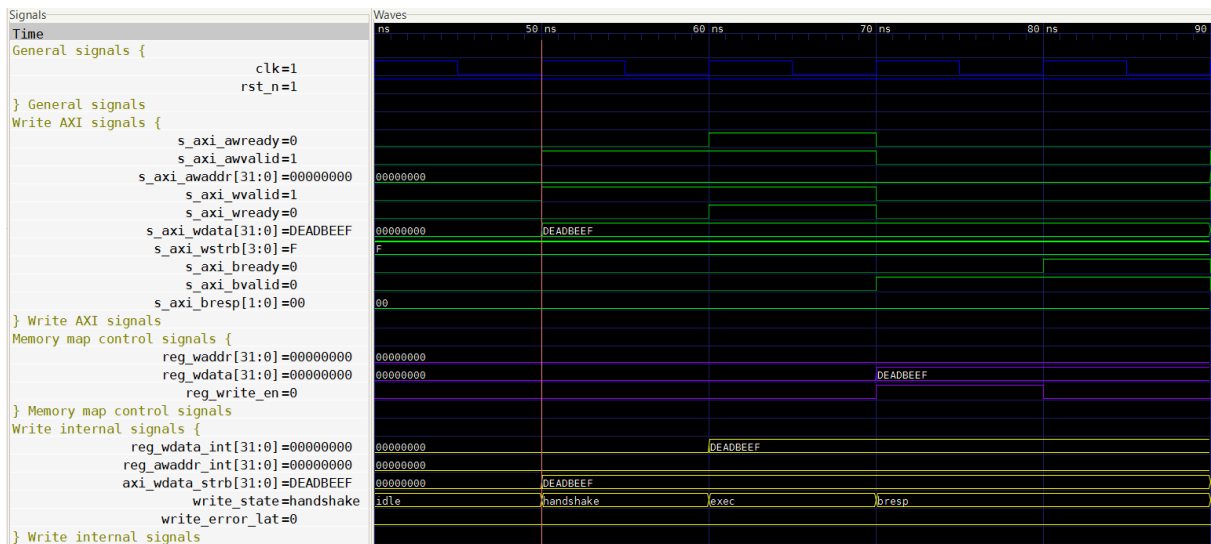


Figure 2 – Test 1: AXI-Lite Write Transaction (Register 0)

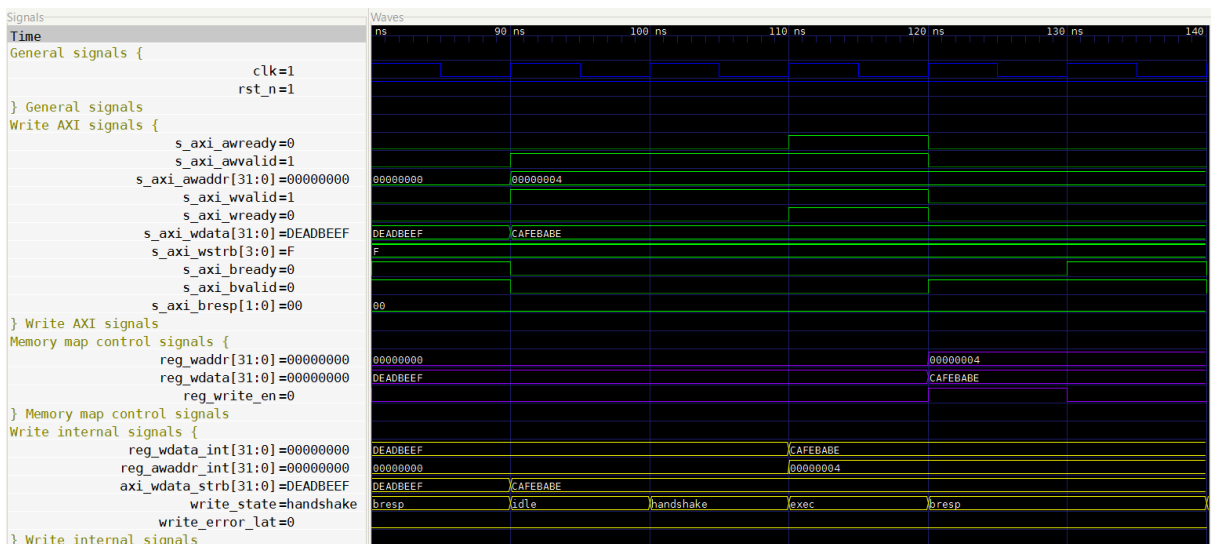


Figure 3 – Test 2: AXI-Lite Write Transaction (Register 1)

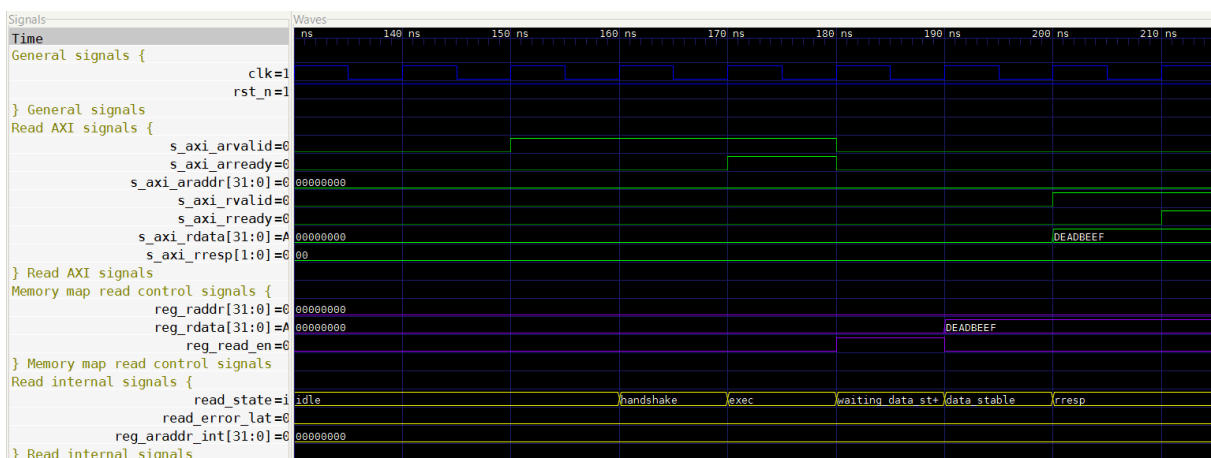


Figure 4 – Test 3: AXI-Lite Read Transaction (Register 0)

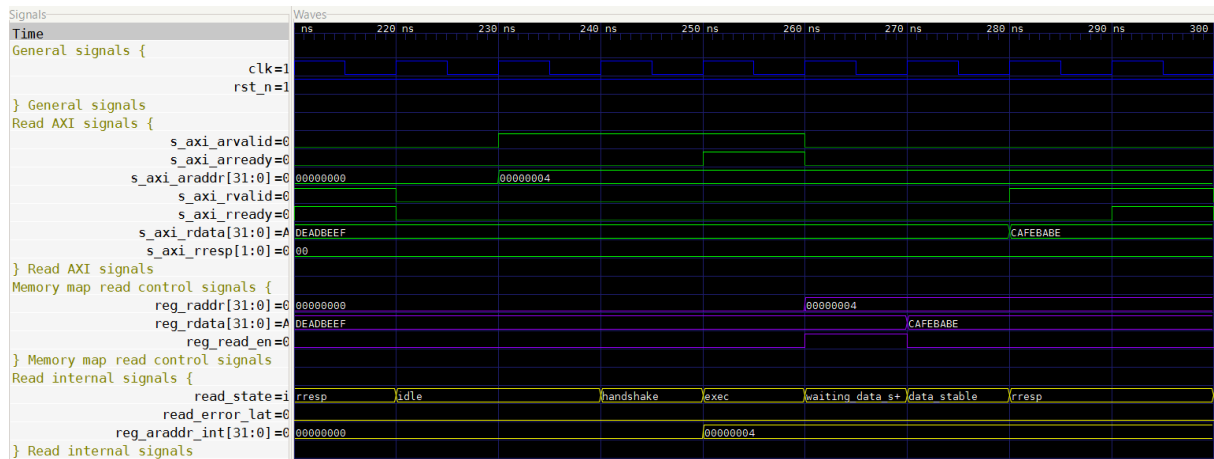


Figure 5 – Test 4: AXI-Lite Read Transaction (Register 1)

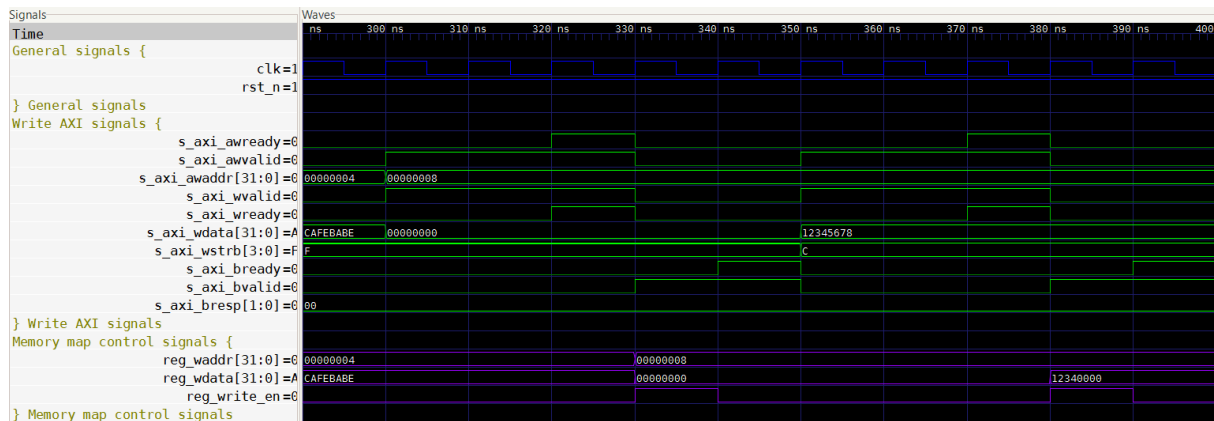


Figure 6 – Test 5: Partial Write Using Byte Strobes

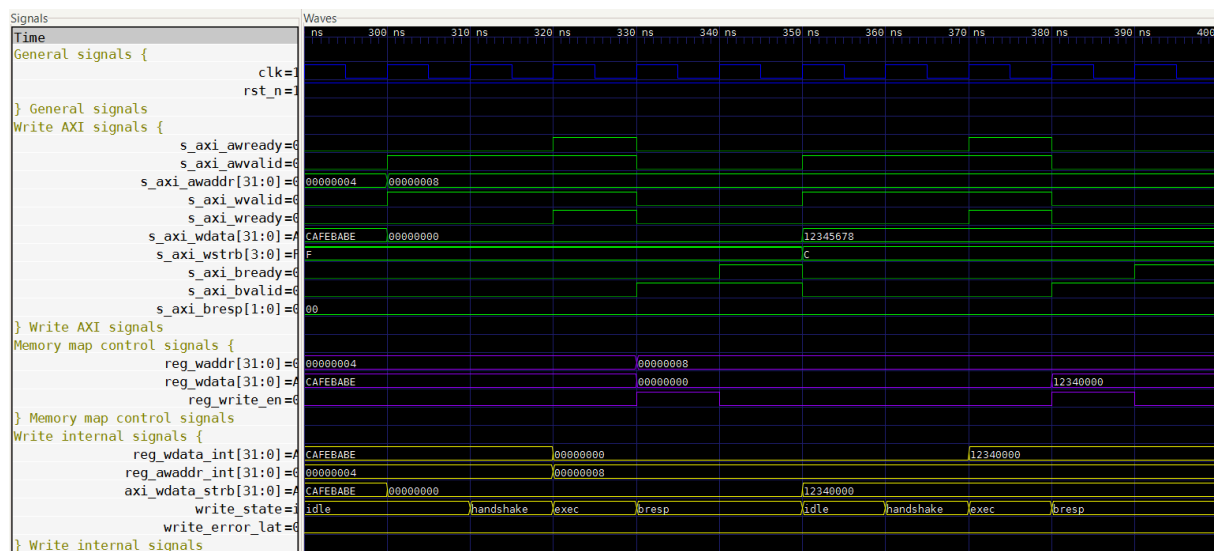


Figure 7 – Test 6: Misaligned Address Error Response

